Multi-Language Benchmark Generation via L-Systems

Vinicius Francisco da Silva
UFMG
Belo Horizonte, Brazil
silva.vinicius@dcc.ufmg.br

Matheus Marchesotti Ferraz
UFMG
Belo Horizonte, Brazil
mmarferraz@gmail.com

Fernando Magno Quintão Pereira
UFMG
Belo Horizonte, Brazil
fernando@dcc.ufmg.br

Abstract

This paper introduces BenchGen, a benchmark synthesizer that uses L-systems (iterated rewriting rules) to generate large, structurally self-similar programs in multiple languages. The goal of BenchGen is to support the evaluation of the performance of language processing systems: compilers, interpreters, static analyzers and such. By exploring notions of self-similarity, BenchGen can generate complex control flow without undefined behavior, and produce families of comparable programs across languages. To demonstrate its effectiveness, this paper reports representative case studies of BenchGen that illustrate how the generated benchmarks can be used to analyze different compilers.

Keywords

L-System, Benchmark, Synthesis

1 Introduction

Language processing systems, such as compilers, interpreters and static analyzers are complex tools whose validation depends on programs written in the target language. However, the number of available benchmarks for any given compiler is typically limited [40]. To address this limitation, several tools have been developed to automatically generate test programs [42]. This process, known as *fuzzing* [27], is widely used to uncover bugs such as crashes and memory leaks. In the compiler domain, fuzzers such as Csmith [42] and YARPGen [26] generate random C programs to stress-test compilers and support static analysis. More recently, fuzzers like Fuzz4All [41] have leveraged Large Language Models (LLMs) to generate test programs not only for compilers but also for constraint solvers, interpreters, and other software systems with accessible APIs.

Despite the abundance of program generators [42], existing tools exhibit important limitations. A key issue is the lack of control over the size of the generated programs. For example, Csmith, the most widely used C compiler fuzzer, does not allow users to tune the output size. Instead, it produces programs whose sizes follow a normal distribution. When compiled with `clang` v9.0.1 at `-O0`, these programs contain on average 20,190 LLVM instructions, with a standard deviation of 3,650 and a median of 19,161 instructions [12]. Other fuzzers, such as YARPGen [26], LDRGen [2], and Orange3 [24], show similar behavior. As a consequence, these tools are ill-suited for performance evaluation of compiler components such as parsing, semantic analysis, and code generation.

Programs via L-Systems: To overcome this limitation, this paper proposes a methodology for stress-testing compilers with a focus on *performance* rather than solely on *correctness.* Our approach enables the controlled generation of programs whose size can be precisely tuned by the user. This makes it possible to construct synthetic code of virtually arbitrary size, constrained only by practical resources such as generation time and storage space.

The central insight is that programs often exhibit recursive, self-similar structures, as discussed in Section 2.1. For instance, the branches of a control construct (e.g., `if-then-else`) are themselves programs that may recursively contain further control constructs. To exploit this property, we introduce a generation technique based on *L-systems* [25]. Originally developed to model the growth of biological organisms, L-systems provide a grammar-based framework that we extend to code generation. This paper shows how families of self-similar programs can be encoded as L-grammars. Programs are then generated by iterative rewriting. Each resulting string encodes the blueprint of a program organized around a central data structure such as an array or a list. As discussed in Section 2, this technique supports multiple programming languages, yields complex control-flow graphs, avoids undefined behavior and enables the manipulation of different data structures.

Summary of Contributions: To demonstrate the benefits of representing program growth with L-systems, we introduce BenchGen, a multi-language benchmark generator. Section 3 evaluates BenchGen through different case studies, including: a comparison between `gcc` and `clang`; a comparison of ten different programming languages; an asymptotic analysis of different components of `clang`; a longitudinal analysis of the evolution of `gcc`; an evaluation of profile-guided optimizations in `clang`; a comparison of data structures from GLib; and a comparison between programs generated by BenchGen and by Csmith.

2 Code Generation via L-Systems

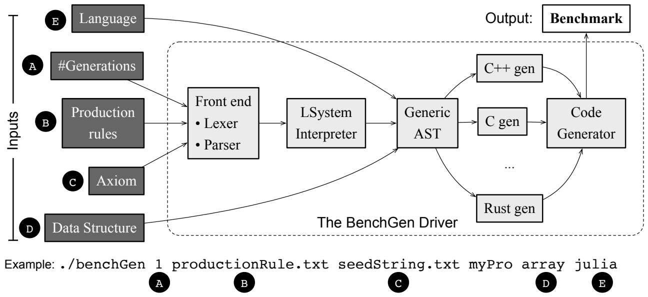
BenchGen generates programs that manipulate data structures according to the schema seen in Figure 1. Section 2.1 introduces the core blocks used in program construction, while Section 2.2 describes semantics of these building blocks.

2.1 Syntactical Building Blocks

The L-systems described in this paper use two families of constructs:

- **Structure:** elements that define the control flow of a program, including `IF`, `LOOP`, and `CALL`.
- **Behavior:** operations that specify how data is manipulated, including `new`, `insert`, `remove`, and `contains`.

2.1.1 Structure Blocks. The control flow in programs generated by BenchGen arises from combining four types of code blocks, as specified by the grammar below:

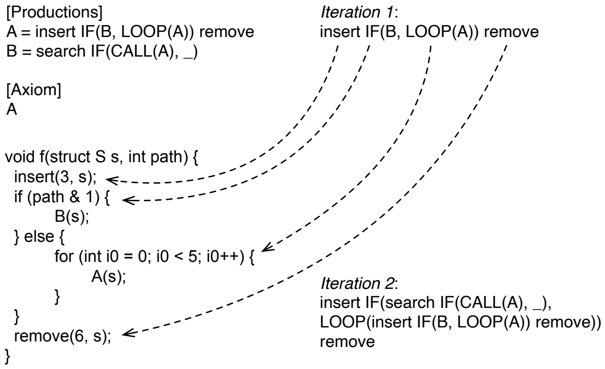


```
Example: ./benchGen 1 productionRule.txt seedString.txt myPro array julia
```

Figure 1: BenchGen is a tool that generates programs in different programming languages. It receives as input the description of an L-System, which consists of a set of production rules, plus a starting symbol (the axiom). It interprets these rules, producing strings (the L-Strings), which guide the construction of genertic ASTs. These trees can be converted to programs in different programming languages using different containers. A code generator then converts the language-specific tree to a program.

$$\begin{array}{lcll} b & ::= & \mathrm{IF}(b_{\mathrm{cond}}, b_{\mathrm{then}}) & ;;\ \textit{if\_then} \\ & | & \mathrm{IF}(b_{\mathrm{cond}}, b_{\mathrm{then}}, b_{\mathrm{else}}) & ;;\ \textit{if\_then\_else} \\ & | & \mathrm{LOOP}(b_{\mathrm{cond}}, b_{\mathrm{body}}) & ;;\ \textit{while} \\ & | & \mathrm{CALL}(b) & ;;\ \textit{function\_call} \end{array}$$

Each of these constructs corresponds to a familiar programming block: conditional branches (`if-then`, `if-then-else`), loops (`while`), and function calls. Figure 2 provides an illustration.



```
[Productions]
A = insert IF(B, LOOP(A)) remove
B = search IF(CALL(A), _)

[Axiom]
A
```

Iteration 1:
insert IF(B, LOOP(A)) remove

```
void f(struct S s, int path) {
  insert(3, s);
  if (path & 1) {
        B(s);
  } else {
        for (int i0 = 0; i0 < 5; i0++) {
              A(s);
        }
  }
  remove(6, s);
}
```

Iteration 2:
insert IF(search IF(CALL(A), _),
LOOP(insert IF(B, LOOP(A)) remove))
remove

Figure 2: This figure shows a grammar designed to synthesize programs. The right-hand side illustrates two derivation steps from this L-grammar, along with a corresponding (simplified) C program produced from the second derivation.

2.1.2 Behavior Blocks. The dynamic behavior of BenchGen programs emerges from interactions with data structures. All data structures within a program must share the same interface. Every language ported to BenchGen must support arrays. Additionally, the C code generator also supports any of the twelve data structures available in the GNOME Library. Adding support for new containers involves extending four C++ classes. These classes define the behavior of four constructs in the L-grammar:

- **new:** Creates and initializes a data structure.
- **insert:** Adds an element to a data structure in scope.
- **remove:** Deletes an element from a data structure in scope.
- **contains:** Checks whether an element exists in a data structure in scope.

Notice that these four behavior blocks can be customized by BenchGen users in any way. For instance, BenchGen provides a "*scalar mode*" to generate programs that manipulate integer variables. In this case, we have configured `insert` to increment variables; `remove` to decrement them; `new` to declare and initialize them with zero; and `contains` to test if they are zero. As a more concrete example, Figure 3 shows how three such operations may be defined for programs working with arrays.

new

```
array_t* array156;
array156 = (array_t*)malloc(sizeof(array_t));
array156->size = 42;
array156->refC = 1;
array156->id = 156;
array156->data = (unsigned int*)malloc(array156->size*sizeof(unsigned int));
memset(array156->data, 0, array156->size*sizeof(unsigned int));
DEBUG_NEW(array157->id);
```

insert

```
unsigned int loop46 = 0;
unsigned int loopLimit46 = (rand()%loopsFactor)/2 + 1;
for(; loop46 < loopLimit46; loop46++) {
  for (int i = 0; i < array156->size; i++) {
    array156->data[i]--;
  }
}
```

constains

```
unsigned int loop46 = 0;
unsigned int loopLimit46 = (rand()%loopsFactor)/2 + 1;
for (int i = 0; i < array156->size; i++) {
  if (array156->data[i] == 61) {
    break;
  }
}
```

Figure 3: Examples of block definitions for `new`, `insert`, and `contains`. This figure shows C code generated to manipulate arrays. Although a program may handle many arrays, the example focuses on a specific variable, `array156`. The operations `new()`, `insert()`, and `contains()` must be customized by the user to define the behavior of the generated code.

2.2 Execution Flow

Programs generated by BenchGen are executable. Their control flow is governed by a variable named `PATH`, of type `unsigned long`, which determines the outcome of conditional branches. Algorithm BitPath in Algorithm 1 correlates the outcome of branches with this `PATH` variable. The algorithm uses a counter stack to ensure each path is uniquely identifiable via the `PATH` variable. It assigns a unique bitmask to each branch condition by:

1. **Tracking Nesting Depth**: Using a stack to manage counters for each level of nested branches.
2. **Resolving Joins**: Propagating the "maximum counter" upward when branches merge, ensuring no bitmask collisions.

The way Algorithm 1 determines how control-flow is taken lets us use BenchGen to test the effectiveness of profile-guided optimizations, as Section 3.5 shows. Figure 4 clarifies this mechanism.

Algorithm 1: BitPath assignment of unique bitmasks to branch conditions

```
Input: A function $f$
Output: A mapping of each branch condition in $f$ to a
        unique bitmask
Initialize the Stack: push a single counter $Counter_0 = 1$,
  representing the least significant bit (LSB);
Assign Masks to Branches:;
    Same Nesting Level: use the current top-of-stack
      counter to generate the bitmask, e.g., PATH & (1 «
      (counter - 1));
    Nested Branch: push a new counter, incremented by 1
      from the parent counter, ensuring fresh bits for deeper
      nesting;
Handle Join Points (after if-then-else):;
    Pop the current counter from the stack;
    Update the parent counter (now on top of the stack) to
      the maximum of:;
        (a) the parent's original value;
        (b) the popped (child) counter;
    This ensures that parent branches account for the
      deepest nesting level beneath them;
Independent Branches: after resolving a join, subsequent
  branches at the same level reuse the updated parent
  counter;
```

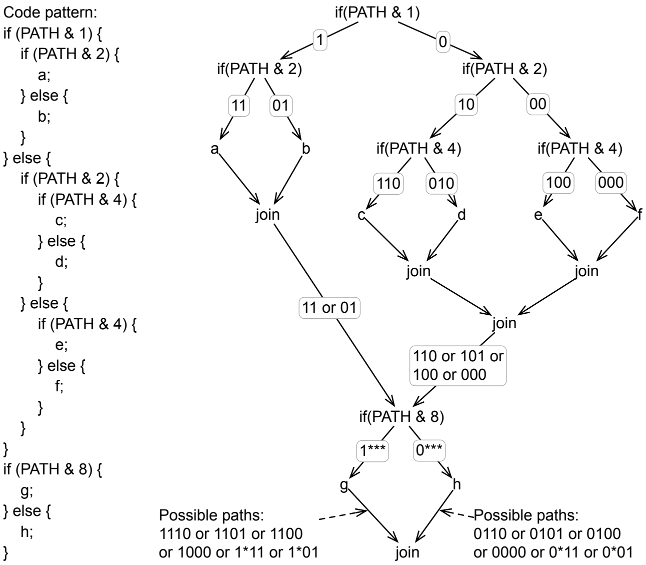
2.3 Function Calls

BenchGen supports function calls through the `CALL` clause. When an L-string includes a construction of the form `CALL`(e), the entire substring e is extracted and defined as a separate function. The functions generated from a given L-specification can be either grouped into a single file or distributed across multiple files, depending on a configuration parameter in BenchGen, as Figure 5 shows. All occurrences of `CALL`(b) with identical strings b result in calls to the same function. To avoid redundancy, BenchGen maintains a table mapping strings to functions, ensuring that identical strings refer to the same function instance.

Parameter Passing. Functions generated by BenchGen receive two parameters:

- **Data:** an array of data structures that enables sharing between caller and callee functions.
- **Path:** a control variable that governs execution flow, as seen in Figure 4.

The **Data** array is populated using a reaching definitions analysis, which determines which variables in the caller function are available to be passed as arguments at the call site. Figure 6 illustrates this mechanism.


Code pattern:

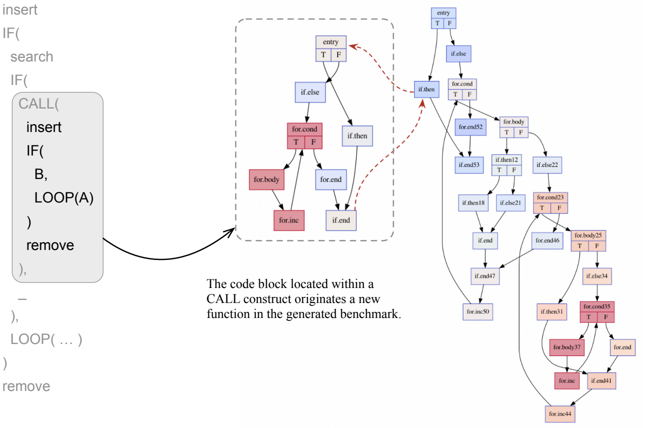
```
if (PATH & 1) {
   if (PATH & 2) {
      a;
   } else {
      b;
   }
} else {
   if (PATH & 2) {
      if (PATH & 4) {
         c;
      } else {
         d;
      }
   } else {
      if (PATH & 4) {
         e;
      } else {
         f;
      }
   }
}
if (PATH & 8) {
   g;
} else {
   h;
}
```

if(PATH & 1)
1
0
if(PATH & 2)
if(PATH & 2)
11
01
10
00
a
b
if(PATH & 4)
if(PATH & 4)
110
010
100
000
join
c
d
e
f
join
join
11 or 01
join
110 or 101 or
100 or 000
if(PATH & 8)
1***
0***
g
h
join

Possible paths:
1110 or 1101 or 1100
or 1000 or 1*11 or 1*01

Possible paths:
0110 or 0101 or 0100
or 0000 or 0*11 or 0*01

Figure 4: The control flow of a synthetic program is determined by the `path` parameter. This figure depicts the control flow graph of a program with the `PATH` variable combined with masks created by the Algorithm 1. Notice that the counter used as the bitmask is updated due to nesting or sequencing. Nesting causes the increment of the counter on the top of the counter stack. Whenever we analyze a branch, its counter will be the maximum of all the possible control-flows that reacher it.



```
insert
IF(
  search
  IF(
   CALL(
    insert
    IF(
     B,
     LOOP(A)
    )
    remove
   ),
   _
  ),
  LOOP( … )
)
remove
```

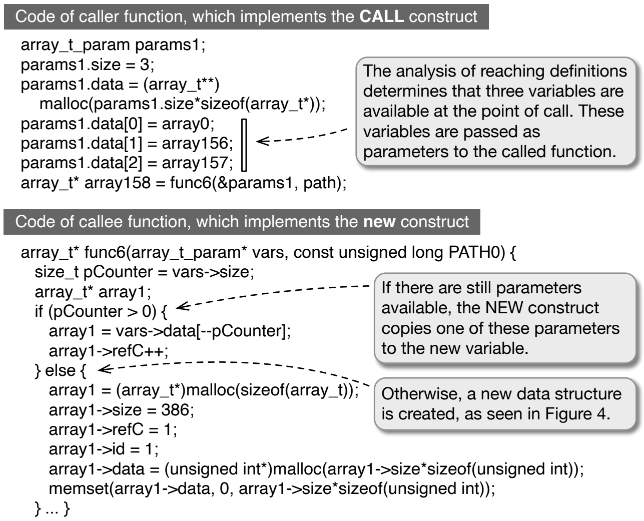
The code block located within a
CALL construct originates a new
function in the generated benchmark.

entry
T
F
if.else
for.cond
for.body
for.inc
if.then
for.end
if.end
entry
if.else
if.then
for.cond
for.end52
for.body
if.end53
if.then12
if.else22
if.then18
if.else21
for.cond23
if.end
for.end46
for.body25
if.end47
if.else34
for.inc50
if.then31
for.cond35
for.body37
for.end
for.inc
if.end41
for.inc44

Figure 5: This figure depicts the control flow produced by a `CALL` block. The enclosed string gives rise to a new function, which becomes part of the synthesized program. This new function is invoked at the point in the L-string where the `CALL` clause appears.

2.4 Memory Management

Programs generated by BenchGen do not suffer from memory leaks, despite relying on heap allocation. To prevent leaks, BenchGen employs a reference-counting garbage collector. This approach



Code of caller function, which implements the **CALL** construct

```
array_t_param params1;
params1.size = 3;
params1.data = (array_t**)
   malloc(params1.size*sizeof(array_t*));
params1.data[0] = array0;
params1.data[1] = array156;
params1.data[2] = array157;
array_t* array158 = func6(&params1, path);
```

The analysis of reaching definitions determines that three variables are available at the point of call. These variables are passed as parameters to the called function.

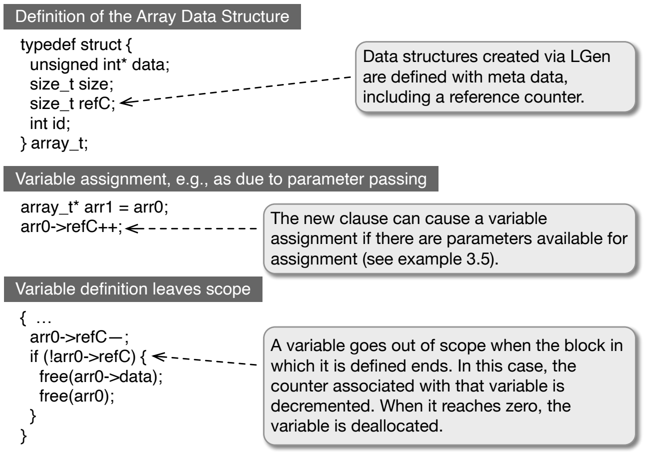
Code of callee function, which implements the **new** construct

```
array_t* func6(array_t_param* vars, const unsigned long PATH0) {
  size_t pCounter = vars->size;
  array_t* array1;
  if (pCounter > 0) {
    array1 = vars->data[--pCounter];
    array1->refC++;
  } else {
    array1 = (array_t*)malloc(sizeof(array_t));
    array1->size = 386;
    array1->refC = 1;
    array1->id = 1;
    array1->data = (unsigned int*)malloc(array1->size*sizeof(unsigned int));
    memset(array1->data, 0, array1->size*sizeof(unsigned int));
  } ... }
```

If there are still parameters available, the NEW construct copies one of these parameters to the new variable.

Otherwise, a new data structure is created, as seen in Figure 4.

Figure 6: Parameter passing in programs created by BenchGen. At the call site, a reaching definitions analysis identifies three variables `array1`, `array156`, and `array157`. Their pointers are stored in `param.data` and passed to `func0`. Inside the caller, each is copied once using `new`; if no parameters remain, `new` creates fresh variables.



Definition of the Array Data Structure

```
typedef struct {
  unsigned int* data;
  size_t size;
  size_t refC;
  int id;
} array_t;
```

Data structures created via LGen are defined with meta data, including a reference counter.

Variable assignment, e.g., as due to parameter passing

```
array_t* arr1 = arr0;
arr0->refC++;
```

The new clause can cause a variable assignment if there are parameters available for assignment (see example 3.5).

Variable definition leaves scope

```
{ ...
  arr0->refC—;
  if (!arr0->refC) {
    free(arr0->data);
    free(arr0);
  }
}
```

A variable goes out of scope when the block in which it is defined ends. In this case, the counter associated with that variable is decremented. When it reaches zero, the variable is deallocated.

Figure 7: This figure illustrates how BenchGen uses reference counting to manage memory. The variables `x` and `y` initially point to two separate heap-allocated structures. Each of these structures contains a `refC` field that holds the number of active references to the object. When the assignment `x := y` occurs, the reference count of the structure originally pointed to by `x` is decremented, as `x` no longer refers to it. If this decrement causes `refC` to reach zero, the structure is deallocated. Meanwhile, the reference count of the structure pointed to by `y` is incremented to account for the new reference from `x`. After the assignment, both `x` and `y` point to the same object, whose `refC` now reflects two active references.

tracks how many references (pointers) exist to each dynamically allocated object. When a reference is created, the count is incremented; when it is removed, the count is decremented. Once the count reaches zero, the object is no longer reachable and can be safely deallocated. To support this mechanism, each structure created by BenchGen includes an additional field, `refC`, which stores the current reference count. Figure 7 shows how reference counting is used in BenchGen to prevent memory leaks from happening.

2.5 Multilanguage Support

BenchGen supports the creation of benchmarks for multiple programming languages. Some of these languages are evaluated in Section 3.2. The benchmark generator itself is implemented in C++. To add support for a new language, users must implement new C++ classes that define the semantics of the structure and behavior blocks described in Section 2.1. This process also requires modifying the templates that generate code for the structure blocks (`IF`, `LOOP`, and `CALL`) and the behavior blocks (`insert`, `remove`, `contains`, and `new`). In addition to implementing new C++ classes, users must register the language module in the main BenchGen driver, which requires changing one line of code in the driver itself. Extending BenchGen therefore involves recompilation. In future work, we aim to fully automate this process and decouple the tool from the specific languages it supports.

3 Experimental Evaluation

This section explores BenchGen's usage in the following *Case Studies*:

- **CS1**: Comparison between `gcc` and `clang` in terms of execution speed, binary size, and compilation time.
- **CS2**: Comparison of the C and C++ compiler backends available in the Gnu Compiler Collection.
- **CS3**: Analysis of the asymptotic behavior of different optimization levels in `clang` and `gcc`.
- **CS4**: Examination of the evolution of `gcc` from version 5 to 14, focusing on execution speed, binary size, and compilation time.
- **CS5**: Evaluation of the impact of profile-guided optimizations in `clang`.
- **CS6**: Comparison of different data structures in the Gnome Library (GLib) with respect to insertion, search, and deletion times.
- **CS7**: Comparison between BenchGen and CSmith, a compiler fuzzer.

Experimental Setup: All experiments were conducted on an Intel(R) Xeon(R) CPU E5-2680 v2 running at 2.80 GHz, with Linux Ubuntu 5.15.0-139-generic. The specific compiler and library versions used are detailed in each case study.

Checking for Undefined Behavior: We analyzed the C programs evaluated in Section 3.1 using four tools capable of detecting undefined behavior: UBSan and ASan from `clang` [38], Valgrind [32], the EVA plugin of Frama-C [4], and KCC [13]. In addition, we verified that every program evaluated in Section 3.2, when executed in debug mode, produces the same trace, regardless of the programming language. While these measures do not formally prove that BenchGen always generates sound benchmarks, they provide strong evidence that this is probable.

3.1 CS1: `gcc` vs `clang`

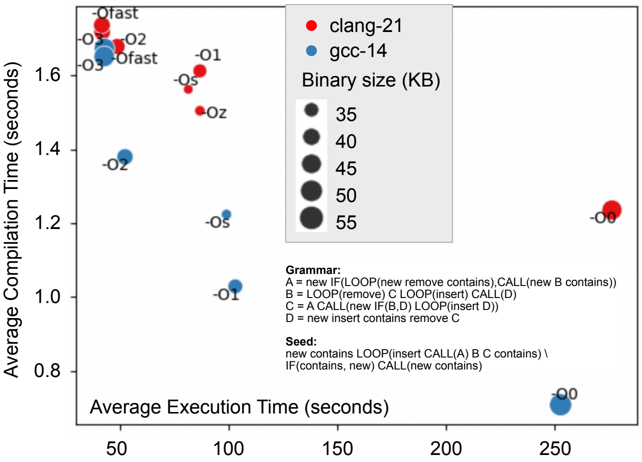
The GNU C Compiler (gcc) and LLVM's `clang` are the two most widely used C compilers, and comparisons between them are common in the literature.

This case study evaluates the GNU C Compiler (`gcc`) and LLVM's `clang` across three performance metrics:

- Execution time of the compiled program.
- Compilation time required to build the program.
- Binary size of the compiled program.

The comparison covers six optimization levels for `gcc` 14.2: `-O0`, `-O1`, `-O2`, `-O3`, `-Os`, and `-Ofast`; and seven for `clang` 21.0: `-O0`, `-O1`, `-O2`, `-O3`, `-Oz`, `-Os`, and `-Ofast`. Binary sizes were collected using the Linux `size` command, considering only the `text` section, which represents the size of the executable code in bytes. Compilation and execution times were measured with `hyperfine`, configured with three warm-up runs and at least ten benchmark runs. To compare the compilers, we have produced nine generations of the same grammar, and report numbers for the ninth.

Discussion. Figure 8 summarizes the results of this comparison. The scatter plots show compilation time against execution time, with point size proportional to binary size. We observe that gcc and `clang` tend to produce programs with similar execution times at the lowest and highest optimization levels; however, `gcc`'s compilation times are generally lower.



```
Grammar:
A = new IF(LOOP(new remove contains),CALL(new B contains))
B = LOOP(remove) C LOOP(insert) CALL(D)
C = A CALL(new IF(B,D) LOOP(insert D))
D = new insert contains remove C

Seed:
new contains LOOP(insert CALL(A) B C contains) \
IF(contains, new) CALL(new contains)
```

Figure 8: Comparison of `gcc` 14.2 and `clang` 21.0 across optimization levels.

A clear trend emerges for `gcc`: it often exhibits a smooth trade-off between compilation time and execution time, forming a Pareto-like curve where longer compilation typically results in faster execution. This pattern is less evident for `clang`. Instead, `clang` shows a marked distinction between the lowest optimization levels (`-O0`, `-O1`) and the higher ones, while the most aggressive optimizations (`-O2`, `-O3`, and `-Ofast`) yield very similar results. This observation echoes findings by Curtsinger and Berger in their work on Coz [11], who reported no statistically significant difference between `-O2` and `-O3` in `clang`[1]. Regarding binary size, both compilers generate smaller executables at `-Os`, with `clang` holding a slight edge (2–3%) at the `-Oz` level. Nevertheless, both can reduce code size by nearly 50% when moving from `-O3` to `-Os`.

3.2 CS2: Comparing Programming Languages

As explained in Section 2.5, BenchGen synthesizes benchmarks in multiple programming languages. This section leverages its multi-language support to compare ten languages: Ada, C, C++, D, Julia, Go, Zig, Nim, V and Odin. To perform this comparison, we used the grammar shown in Figure 8 to produce eleven generations of a program, and report five runs of the 11th. Programs in each language run with the latest version of each runtime environment available on May 26th, 2026. Julia runs in interpreted mode with just-in-time compilation, whereas the other languages are compiled ahead of time. V has two execution modes: native and C-backend. The former directly compiles V to machine code, whereas the latter first translates V to C, and then uses `clang-22` to produce binary code out of this C source. In this case, we use the optimization level `-O3`, as we do in C and C++. We include two versions of C++, one using plain arrays and another using `std::vector`, just to give the reader some perspective on the relative running times that Figure 9 presents. Ada runs in two modes: safe and unsafe. The latter disables integer overflow and array bound checks (via the `-gnatp` flag). Each language produces the same number of source files, except for ADA, C, and C++. The latter two generate one extra header file, whereas ADA produces two files per function. For example, in the 11th generation, BenchGen produces 52 files for Julia and Go, and 53 for C and C++ and 133 files for ADA.

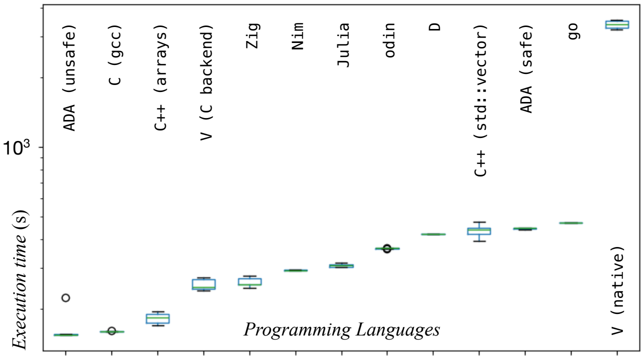


Figure 9: Execution time of benchmarks produced by BenchGen in different programming languages.

Discussion. Figure 9 compares execution time across languages. The results align with expectations: they highlight the performance differences between low-level, statically compiled languages (C and C++) and higher-level languages with different compilation models (Go and Julia). Although we omit the running time of programs in generations 1 to 10, we emphasize that all languages exhibit an exponential increase in execution time as program depth grows. This behavior is inherent to the fractal-based benchmark generator.

---

[1] See https://youtu.be/r-TLSBdHe1A?t=1438 at 23:58

The trends observed in Figure 9 are consistent with those documented on the well-known “Benchmark Game” website[2]. A key distinction, however, lies in the implementation strategy. While the Benchmark Game compares hand-optimized solutions that may differ syntactically and semantically, BenchGen produces structurally similar code across languages. This methodology offers a more precise comparison of compiler performance by eliminating variability introduced by differing algorithmic implementations.

3.3 CS3: Asymptotic Behavior

The ability to gradually vary the size of programs generated by BenchGen makes it a suitable tool for conducting empirical asymptotic analyses of language processing systems. In this section, we illustrate this capability by empirically evaluating the asymptotic complexity of the full compilation pipeline of gcc. For this purpose, we employ a standard L-System to produce eight generations of a program (from the 5th to the 12th) and feed these programs to the compilers.

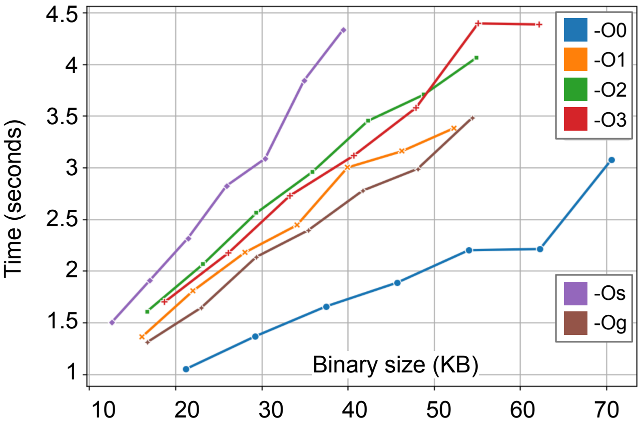


Figure 10: Empirical evaluation of the asymptotic behavior of different compilation phases of `clang` and of the full compilation pipeline of `gcc`.

Discussion. Figure 10 relates the running time of the different compilation modes of gcc to the size of the binary processed at each generation of the target program. All analyzed modes exhibit linear behavior for large programs, with very strong correlations (above 0.9) across different optimization levels. However, the constant factors vary considerably. The size-optimization level (`-Os`) exhibits the steepest growth, which is expected since `gcc -Os` produces the smallest binaries. Overall, these results suggest that, at least when compiling BenchGen programs, gcc does not exhibit asymptotic performance bugs of the kind reported in previous work for other tools [18, 33].

3.4 CS4: The Evolution of gcc

This case study compares different versions of gcc across multiple optimization levels using three metrics:

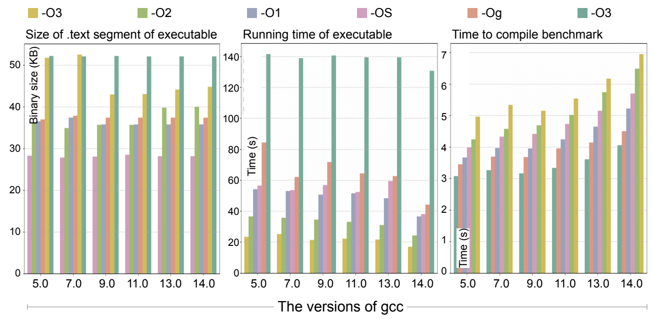


Figure 11: Evolution of gcc from version 5 to 14 across six optimization levels. Results are shown for (left) binary size of the `.text` segment, (center) execution time of the benchmark, and (right) compilation time.

- Execution time of the compiled program (`hyperfine`).
- Compilation time required to build the program (`hyperfine`).
- Size of the `.text` segment of the executable (Linux `size`).

For this comparison, we used BenchGen with the first L-System shown in Figure 8. The program generated at the 10th iteration of this grammar was compiled with six versions of gcc.

Discussion. Figure 11 summarizes results for six optimization levels: `-O0`, `-Og`, `-O1`, `-Os`, `-O2`, and `-O3`. Each group of bars corresponds to a compiler version: `gcc5`, `gcc7`, `gcc9`, `gcc11`, `gcc13`, and `gcc14`. Below we discuss some of the insights of this study:

Compilation time: We observe a steady increase in compilation time with each new version of gcc, particularly at higher optimization levels such as `-O3`. Comparing versions 5 and 14, compilation time rose by about 42% over nine years, with a 31% increase even at `-O0`. This is unsurprising: new optimization passes have been added over time, which increases compile time in exchange for better code quality.

Execution time: Performance has improved at the higher optimization levels. Comparing `gcc5` to `gcc14`, execution time decreased by roughly 27% at `-O3`, 32% at `-O1`, and as much as 47% at `-Og`. The gains at `-O0` were modest, about 7%. These results indicate that newer versions of gcc generate more efficient code, especially at intermediate optimization levels.

Binary size: Binary size has remained stable across versions. At `-Os`, code size decreased only 0.34% between `gcc5` and `gcc14`. At `-O3`, binary size initially increased from `gcc5` until `gcc7`, dropping noticeably in `gcc9`. Past this point, we again observe a small increase until `gcc14`. However, in this version code is still almost 10% smaller than in `gcc5`.

Over the evolution from version 5 to 14, gcc trades longer compilation times in exchange for more efficient executables. While binary size has changed little, execution performance has improved noticeably, particularly at `-Og` and `-O3`. These results indicate that the gcc community has consistently prioritized code quality and runtime efficiency over compilation speed, accepting longer build times in return for measurable performance gains.

---

[2] Available at https://benchmarksgame-team.pages.debian.net/benchmarksgame/ as of September 20th.

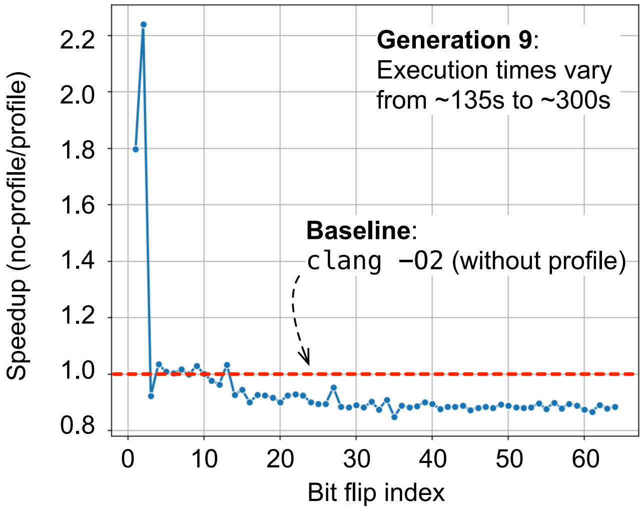


Figure 12: Impact of profile guided optimizations on `clang`.

3.5 CS5: Profile Guided Optimizations in `clang`

Profile-Guided Optimizations (PGO) leverage runtime information collected from representative executions of a program to improve the quality of its code. In `clang`, PGO serves as a refinement to existing optimization levels, most notably `-O2` and `-O3`. When profile data is provided via the `-fprofile-use` flag, `clang` applies the chosen baseline optimizations but augments them with profile-driven insights, generating code that more accurately reflects the observed runtime behavior of the program. This section illustrates how BenchGen can be used to evaluate the effectiveness of profile-guided optimizations (PGO) using the following methodology to modify the execution flow discussed in Section 2.2:

1. Generate four program variants (Generations 6, 7, 8, and 9) using the following grammar with array structures:
```
A = new B B
B = IF(LOOP(insert A contains), LOOP(insert A contains))
```
2. Compile each program at the `-O2` optimization level with PGO enabled.
3. During the "*Training Phase*," collect profile information by running each program with PATH $= 2^0$ (i.e., PATH $= 1$).
4. During the "*Testing Phase*," for $i = 1$ to 63:
   (a) Execute each program with PATH $= 2^i - 1$, e.g., PATH $= 0b1$, PATH $= 0b11$, etc.

Let t be the execution time of the program compiled with `clang -O2` without profile, and let t_i be the execution time with PATH $= 2^i$. The ratio t/t_i quantifies the relative speedup due to profile-guided optimizations as the control-flow path diverges by i bits.

Discussion. Figure 12 shows the data collected during this experiment for one of our synthetic benchmarks. These results reveal two distinct behaviors of profile-guided optimizations. When the execution path during testing closely resembles the path exercised during training, we observe significant speedups, often reaching 2x. Similar results on real applications have been reported in previous work [8, 28, 35].

As the number of bit flips in PATH increases, however, the executed control flow diverges from the profiled run, and the benefit of PGO diminishes. Eventually, for highly divergent inputs, the profile-optimized binary becomes slower than its non-profiled counterpart. This behavior suggests an instance of profile overfitting: because PGO biases optimizations toward the observed hot path, code along unprofiled paths may suffer from unfavorable decisions. By reading `perf` counters, we observe that the program optimized with profile data experiences more cache references and cache misses than the program optimized without it, as the training and test path diverge. The ratio of branch misses, in contrast, do not show more variance as the paths diverge. A plausible explanation for this effect is the difference in the number of functions inlined with and without profiling data.

3.6 CS6: Comparison with CSMITH

Several programming language fuzzers enjoy widespread popularity. The main goal of such tools, including CSmith [42] and **YarpGen** [26] (for C) or ChiGen [39] (for Verilog), is to uncover bugs in language processing systems such as compilers or interpreters. The purpose of BenchGen, however, is different: it was designed to evaluate the performance of these systems. Its ability to expose bugs is limited, since the programs it produces are less diverse than those generated by a fuzzer like CSmith. On the other hand, BenchGen can stress-test compiler performance in ways that CSmith and related tools cannot. This section illustrates this point through the following challenge: generate a collection of programs whose execution time is comparable to the average runtime of Spec Cpu2017 programs, and whose behavior under optimization resembles that of real-world workloads.

Discussion. To compare CSmith and BenchGen under this challenge, we have used these two program generators to produce a collection of four benchmarks and compare them with the programs from Spec Cpu2017. The adopted methodology is described below:

1. We ran the 28 Spec Cpu2017 programs that `clang`/LLVM 20.1 can compile.
2. Using CSmith, we generated 10,000 programs and selected the four whose execution time was closest to the average runtime of the 28 Spec Cpu2017 programs compiled without optimizations.
3. Using the four grammars described in Section 3.1, we generated four BenchGen programs whose runtime approximated the same average.
4. We then optimized each collection of benchmarks (28, 4, and 4 programs, respectively) under different optimization flags.

Table 1 presents the results of this experiment. We stopped BenchGen as soon as we obtained programs whose running time was above the average Spec time, and chose the generation immediately before. However, it is not possible to steer CSmith in such a way. In other words, users cannot control CSmith to produce programs that execute within a given time frame. Most CSmith programs tend to run for only a very short time. Moreover, the impact of optimizations on these programs differs substantially from what we observe in real-world benchmarks such as those in the Spec collection, whereas the effects observed on BenchGen programs more closely resemble the behavior of genuine workloads.

	-O0	-O1	-O2	-O3
BenchGen				
Cycles (Mean)	1.44E+11	5.21E+10	5.20E+10	5.14E+10
Ratio	1	2.76	2.77	2.80
SPEC C2017				
Cycles (Mean)	5.63E+11	2.43E+11	2.37E+11	2.30E+11
Ratio	1	2.32	2.37	2.44
CSmith				
Cycles (Mean)	2.85E+06	1.57E+06	1.49E+06	1.44E+06
Ratio	1	1.85	1.91	1.98

Table 1: Mean cycles and ratios for different benchmarks across optimization levels.

This difference between the effects of compiler optimizations on CSmith and real-world programs had been noticed in previous work [12].

4 Related Work

BenchGen is a tool designed to synthesize benchmarks. A number of works in the compiler literature have also proposed techniques for benchmark generation. In what follows, we review this body of work, emphasizing aspects that make BenchGen unique.

Compiler Fuzzers. Most existing compiler fuzzers are designed to uncover bugs in language processing systems, whereas BenchGen is specifically aimed at analyzing their performance. Consequently, BenchGen would be less effective than a fuzzer such as CSmith for identifying errors in C compilers. On the other hand, it enables tasks that are not easily supported by traditional fuzzers, such as studying the asymptotic behavior of compiler optimizations, comparing compilers in terms of compilation time, code size, and execution speed, or analyzing the evolution of a single compiler across multiple versions with respect to these same metrics.

Several tools are able to generate random strings from the production rules of a context-free grammar [5, 21, 39, 43]. Indeed, the core ideas behind grammar-guided fuzzing date back to the 1970s [36]. A well-known limitation of this approach, however, is that for programming languages it is difficult to guarantee that the generated strings correspond to executable or even compilable programs. Even fuzzers explicitly designed to generate executable programs may fail to produce compilable code. For example, Ricardo *et al.* [37] report a compilation success rate of 78.49% when using YarpGen [26] to generate C/C++ programs, while Vieira *et al.* [39] report a success rate of approximately 60% when using the ChiGen fuzzer to produce Verilog designs. This limitation is consistent with our experience in developing BenchGen, where a substantial portion of the engineering effort was devoted to ensuring that every generated program compiles without warnings and executes without runtime exceptions.

Benchmark Synthesizers. The development of compilers requires *benchmarks.* For this reason, some of the most celebrated papers in programming languages describe benchmark suites, such as Spec CPU2006 [20], MiBench [17], Rodinia [7], etc. These benchmarks are manually curated and usually comprise a small number of programs. A few years ago, Cummins et al. [9] demonstrated that this small size fails to cover the space of program features that a compiler is likely to explore during its lifetime.

The generation of benchmarks for tuning predictive compilers has been an active research field over the last ten years. Early efforts aimed at the development of predictive optimizers used synthetic benchmarks designed to find compiler bugs. Examples of such synthesizers include CSmith [42], LDRGen [2], and Orange3 [30, 31]. Although conceived as test case generators, these tools have also been used to improve the quality of optimized code emitted by mainstream C compilers [3, 19]. Even CompilerGym [10], a tool for applying machine-learning-based code optimization techniques, provides randomly produced CSmith programs. However, more recent developments indicate that synthetic codes tend to poorly reflect the behavior of programs written by humans; consequently, they produce deficient training sets [12, 15]. Section 3.6 discusses precisely these issues with CSmith: the difficulty of using it to generate programs that run for a reasonable amount of time and the ease with which compilers optimize such programs.

In order to produce more realistic benchmarks, a vast literature on extracting programs from repositories has recently emerged, seeking to build benchmark suites for training compilers. Some of these works aim to generate benchmarks to feed machine learning models [14, 16, 34], for example. This literature has some shortcomings when compared to BenchGen:

- Very large benchmark suites, such as AnghaBench [12] or Poj [29], compile but do not run.
- Benchmark suites designed to run sometimes exhibit undefined behavior, such as ExeBench [1] and ColaGen [6].
- Techniques that generate benchmarks using LLMs, such as the system of Italiano and Cummins [22], fail to produce large programs and take a long time to generate even a small number of programs, as recently shown by Guimarães *et al* [37].
- The Jotai collection [23], which was built from programs that run without undefined behavior, contains only very small programs. In our attempt to run the programs available in CompilerGym [10], the longest-running program took only 0.017 seconds when compiled with `clang -O0`.

5 Conclusion

This paper has presented a methodology for generating benchmarks based on the observation that programs often exhibit self-similar structures, and thus can be described as derivations of L-Systems. We validated this idea through the design and implementation of BenchGen, a tool capable of producing benchmarks in multiple programming languages. In contrast to typical fuzzers, BenchGen is multilingual and allows users to generate programs of arbitrary size whose execution exercises complex control-flow patterns.

Future Work. Beyond the concrete implementation of BenchGen, the key contribution of this paper is the proposal that programs can be systematically generated as instances of L-Systems. The particular L-System flavor explored here employs a combination of seven constructs (three control-flow structures and four data-structure behaviors). Nevertheless, the approach can be extended to a much broader set of syntactic features. For example, the current version of BenchGen generates programs restricted to a single data structure at a time, whereas nothing in the formalism prevents richer combinations of data structures. Likewise, additional control-flow constructs such as `switch`, `do-while`, or even `goto` could be naturally incorporated. We leave the investigation of such extensions as promising directions for future work.

ARTIFACT AVAILABILITY

The authors declare that the research artifacts supporting the findings of this study are accessible at:

- https://github.com/viniciusfdasilva/benchgen-artifact
- https://github.com/viniciusfdasilva/benchgen-artifact-data

BenchGen is available at https://github.com/lac-dcc/BenchGen, under the Apache-2.0 license.

Acknowledgment

This project was supported by Google and by FAPEMIG (Grant APQ-00440-23). We are grateful to Xinliang (David) Li and Victor Lee for their efforts in making the Google sponsorship possible. Lucas Victor Silva produced the data in Table 1. Bruno Pena Baêta, Heitor Leite and Matheus Alcântara worked on an earlier version of BenchGen.

References

[1] Jordi Armengol-Estapé, Jackson Woodruff, Alexander Brauckmann, José Wesley de Souza Magalhães, and Michael F. P. O'Boyle. Exebench: an ml-scale dataset of executable c functions. In *MAPS*, page 50–59, New York, NY, USA, 2022. Association for Computing Machinery.

[2] Gergö Barany. Liveness-driven random program generation. In *LOPSTR*, pages 112–127, Heidelberg, Germany, 2017. Springer.

[3] Gergö Barany. Finding missed compiler optimizations by differential testing. In *Proceedings of the 27th International Conference on Compiler Construction*, CC 2018, pages 82–92, New York, NY, USA, 2018. ACM.

[4] Patrick Baudin, François Bobot, David Bühler, Loïc Correnson, Florent Kirchner, Nikolai Kosmatov, André Maroneze, Valentin Perrelle, Virgile Prevosto, Julien Signoles, and Nicky Williams. The dogged pursuit of bug-free c programs: the frama-c software analysis platform. *Commun. ACM*, 64(8):56–68, July 2021.

[5] Bachir Bendrissou, Cristian Cadar, and Alastair F. Donaldson. Grammar mutation for testing input parsers. *ACM Trans. Softw. Eng. Methodol.*, 34(4), April 2025.

[6] Maksim Berezov, Corinne Ancourt, Justyna Zawalska, and Maryna Savchenko. COLA-Gen: Active Learning Techniques for Automatic Code Generation of Benchmarks. In Francesca Palumbo, João Bispo, and Stefano Cherubin, editors, *PARMA-DITAM*, volume 100 of *Open Access Series in Informatics (OASIcs)*, pages 3:1–3:14, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[7] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *IISWC*, pages 44–54, Washington, DC, USA, 2009. IEEE.

[8] Dehao Chen, David Xinliang Li, and Tipp Moseley. Autofdo: automatic feedback-directed optimization for warehouse-scale applications. In *CGO*, page 12–23, New York, NY, USA, 2016. Association for Computing Machinery.

[9] Chris Cummins, Pavlos Petoumenos, Zheng Wang, and Hugh Leather. Synthesizing benchmarks for predictive modeling. In *CGO*, pages 86–99, Piscataway, NJ, USA, 2017. IEEE.

[10] Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, Jia Liu, Olivier Teytaud, Benoit Steiner, et al. Compilergym: Robust, performant compiler optimization environments for ai research. In *CGO*, pages 92–105, New York, USA, 2022. IEEE.

[11] Charlie Curtsinger and Emery D. Berger. Coz: finding code that counts with causal profiling. In *SOSP*, page 184–197, New York, NY, USA, 2015. Association for Computing Machinery.

[12] Anderson Faustino da Silva, Bruno Conde Kind, José Wesley de Souza Magalhães, Jerônimo Nunes Rocha, Breno Campos Ferreira Guimarães, and Fernando Magno Quintão Pereira. AnghaBench: A suite with one million compilable C benchmarks for code-size reduction. In *CGO*, pages 378–390, Los Alamitos, CA, USA, 2021. IEEE.

[13] Chucky Ellison and Grigore Rosu. An executable formal semantics of c with applications. *SIGPLAN Not.*, 47(1):533–544, January 2012.

[14] Jaroslav Fowkes and Charles Sutton. Parameter-free probabilistic api mining across github. In *FSE*, page 254–265, New York, NY, USA, 2016. Association for Computing Machinery.

[15] Andrés Goens, Alexander Brauckmann, Sebastian Ertel, Chris Cummins, Hugh Leather, and Jeronimo Castrillon. A case study on machine learning for synthesizing benchmarks. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, pages 38–46, New York, NY, USA, 2019. ACM.

[16] Georgios Gousios and Diomidis Spinellis. Mining software engineering data from github. In *ICSE-C*, page 501–502, Washington, DC, US, 2017. IEEE Press.

[17] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown. MiBench: A free, commercially representative embedded benchmark suite. In *WWC*, pages 3–14, Washington, DC, USA, 2001. IEEE.

[18] Xue Han and Tingting Yu. An empirical study on performance bugs for highly configurable software systems. In *ESEM*, New York, NY, USA, 2016. Association for Computing Machinery.

[19] Atsushi Hashimoto and Nagisa Ishiura. Detecting arithmetic optimization opportunities for c compilers by randomly generated equivalent programs. *IPSJ Transactions on System LSI Design Methodology*, 9:21–29, 2016.

[20] John L. Henning. SPEC CPU2006 benchmark descriptions. *SIGARCH Comput. Archit. News*, 34(4):1–17, September 2006.

[21] Renáta Hodován, Ákos Kiss, and Tibor Gyimóthy. Grammarinator: a grammar-based open source fuzzer. In *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, A-TEST 2018, page 45–48, New York, NY, USA, 2018. Association for Computing Machinery.

[22] Davide Italiano and Chris Cummins. Finding missed code size optimizations in compilers using large language models. In *International Conference on Compiler Construction*, page 81–91, New York, NY, USA, 2025. Association for Computing Machinery.

[23] Cecilia Conde Kind, Michael Canesche, and Fernando Magno Quintao Pereira. Jotai: a methodology for the generation of executable c benchmarks. Technical report, Technical Report 02-2022. Universidade Federal de Minas Gerais, 2022.

[24] Kota Kitaura and Nagisa Ishiura. Random testing of compilers' performance based on mixed static and dynamic code comparison. In *Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*, A-TEST 2018, page 38–44, New York, NY, USA, 2018. Association for Computing Machinery.

[25] Aristid Lindenmayer. Mathematical models for cellular interactions in development i. filaments with one-sided inputs. *Journal of theoretical biology*, 18(3):280–299, 1968.

[26] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. Random testing for c and c++ compilers with yarpgen. *Proc. ACM Program. Lang.*, 4(OOPSLA), November 2020.

[27] Valentin J.M. Manes, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering*, 47(11):2312–2331, 2021.

[28] Angélica Aparecida Moreira, Guilherme Ottoni, and Fernando Magno Quintão Pereira. Vespa: static profiling for binary optimization. *Proc. ACM Program. Lang.*, 5(OOPSLA), October 2021.

[29] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. Convolutional neural networks over tree structures for programming language processing. In *AAAI*, page 1287–1293, Palo Alto, CA, US, 2016. AAAI Press.

[30] Eriko Nagai, Atsushi Hashimoto, and Nagisa Ishiura. Reinforcing random testing of arithmetic optimization of c compilers by scaling up size and number of expressions. *IPSJ Trans. System LSI Design Methodology*, 7:91–100, 2014.

[31] Kazuhiro Nakamura and Nagisa Ishiura. Introducing loop statements in random testing of c compilers based on expected value calculation, 2015.

[32] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *PLDI*, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.

[33] Oswaldo Olivo, Isil Dillig, and Calvin Lin. Static detection of asymptotic performance bugs in collection traversals. In *PLDI*, page 369–378, New York, NY, USA, 2015. Association for Computing Machinery.

[34] David N Palacio, Alejandro Velasco, Daniel Rodriguez-Cardenas, Kevin Moran, and Denys Poshyvanyk. Evaluating and explaining large language models for code using syntactic structures, 2023.

[35] Maksim Panchenko, Rafael Auler, Bill Nell, and Guilherme Ottoni. Bolt: a practical binary optimizer for data centers and beyond. In *CGO*, page 2–14. IEEE Press, 2019.

[36] Paul Purdom. A sentence generator for testing parsers. *BIT*, 12(3):366–375, September 1972.

[37] Gabriel Ricardo, Natanael Santos Junior, Flavio Figueiredo, and Fernando Pereira. On the practicality of llm-based compiler fuzzing. In *SBLP*, pages 59–66, Porto Alegre, RS, Brasil, 2025. SBC.

[38] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. Addresssanitizer: a fast address sanity checker. In *USENIX ATC*, page 28, USA, 2012. USENIX Association.

[39] João Victor Amorim Vieira, Luiza de Melo Gomes, Rafael Sumitani, Raissa Maciel, Augusto Mafra, Mirlaine Crepalde, and Fernando Magno Quintão Pereira. Bottom-up generation of verilog designs for testing eda tools, 2025.

[40] Zheng Wang and Michael F. P. O'Boyle. Machine learning in compiler optimization. *Proceedings of the IEEE*, 106(11):1879–1901, 2018.

[41] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery.

[42] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in c compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, page 283–294, New York, NY, USA, 2011. Association for Computing Machinery.

[43] José Antonio Zamudio Amaya. Shaping test inputs in grammar-based fuzzing. In *ISSTA*, page 1901–1905, New York, NY, USA, 2024. Association for Computing Machinery.